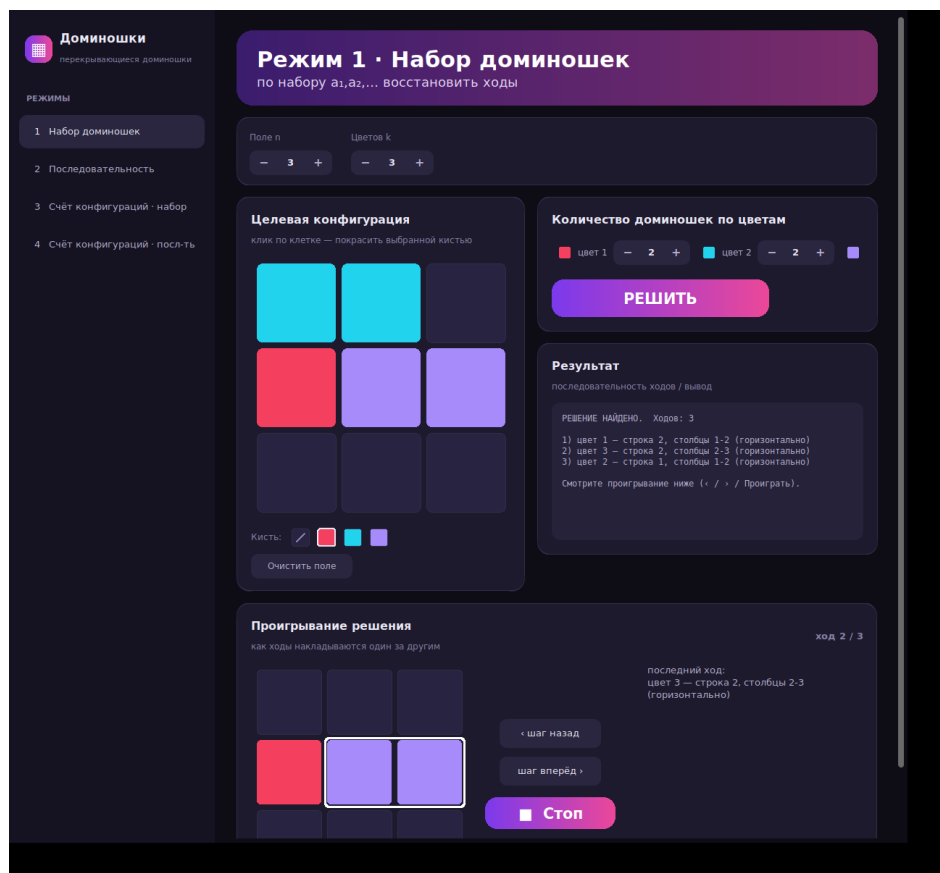


Перекрывающиеся доминошки

Решение задачи

Автор: Гарбунович Иван

ФПМИ БГУ, специальность «Информатика», 1 группа
проект на поступление



Так выглядит программа (режим 1).

1. О чём задача

Есть квадратное поле. На него по очереди кладут цветные доминошки: каждая занимает две соседние клетки и кладётся либо вдоль, либо поперёк. Класть разрешено внахлёт, поэтому если одна доминошка легла на другую, видна будет та, что легла позже. Клетка, до которой никто не дотянулся, остаётся пустой.

Дальше показывают только готовую картинку - то, что получилось в итоге. По ней требуется понять, в каком порядке доминошки клали. Если такого порядка не существует, нужно объяснить, почему. Ещё в задаче требуется посчитать, сколько разных картинок вообще может получиться.

В условии четыре вопроса, и в проекте сделаны все четыре, а вместе с ними - окно с картинкой, версия для консоли и набор проверок.

Четыре режима

- **Режим 1.** Задано, сколько доминошек какого цвета есть, и показана картинка. Нужно найти порядок ходов или объяснить, что его нет.
- **Режим 2.** Задана готовая последовательность цветов - менять её нельзя, класть надо все доминошки. Нужно расставить их по местам или объяснить, почему это невозможно.
- **Режим 3.** Задан набор доминошек. Сколько разных картинок из него можно сложить?
- **Режим 4.** То же самое, но порядок цветов известен заранее.

2. Как устроено поле

Поле - это таблица чисел. В каждой клетке стоит:

- ноль, если клетка пустая, то есть её никто не накрывал;
- номер цвета, если клетку накрыли, причём цвет той доминошки, которая легла последней.

Ход - это «положить доминошку такого-то цвета на эти две соседние клетки». Раскладка - список ходов по порядку. Если разложить ходы по очереди на пустом поле, получится картинка: в каждой клетке остаётся цвет того хода, который накрыл её последним.

3. Главная мысль: разбирать нужно с конца

Первое, что приходит в голову, - перебирать, какую доминошку положили первой, какую второй и так далее. Вариантов при этом получается столько, что программа считает бесконечно.

Поэтому разбор идёт в обратную сторону. Вопрос ставится так: **какую доминошку положили последней?** Найти её легко. Раз она легла последней, её никто не закрасил, значит обе её клетки на картинке показывают её цвет - она видна целиком.

А раз видна, её можно убрать с картинки. Клетки, которые она занимала, помечаются как разобранные: какого цвета они были до неё, уже не важно, ведь сверху лежала именно она. Дальше то же самое повторяется с предпоследней доминошкой, потом с той, что перед ней, и так пока цветных клеток не останется.

Когда картинка разобрана, ответ получается сам собой: **порядок выкладывания - это порядок разбора, прочитанный наоборот**. Разбор шёл с конца, значит клали в обратную сторону.

Осталось точно сказать, когда доминошку разрешено убирать. Убрать её можно, если выполнены три условия:

- под ней нет ни одной пустой клетки(фона);
- все клетки под ней, которые ещё не разобраны, одного цвета;
- хотя бы одна такая неразобранная клетка там есть, иначе убирается пустое место и толку от этого нет.

4. Почему это работает

Дальше объясняется, почему такой разбор работает.

Наблюдение 1. Убирать доминошки соседям только на пользу

Пусть какую-то доминошку сейчас можно убрать, но вместо неё убрали другую, в стороне. Не испортился ли от этого первый ход?

Не испортился. При снятии клетки только помечаются как разобранные. Новых цветов при этом не появляется, пустых клеток тоже не появляется. Значит у первой доминошки список неразобранных клеток мог только сократиться, а их цвета остались прежними - то есть по-прежнему все одного цвета. Пустых клеток под ней как не было, так и не будет. Убрать её по-прежнему можно, пока под ней есть что убирать.

Короче говоря, разобранный клетка никому не мешает: она подходит под любой цвет. Поэтому чем больше разобрано, тем свободнее становится.

Наблюдение 2. На пустую клетку класть нельзя

Пустая клетка на картинке - это клетка, которую вообще никто не накрывал. Если бы её хоть раз накрыли, на ней остался бы цвет той доминошки, что накрыла её последней. Значит ни один ход не задевает пустые клетки, и любая доминошка целиком лежит на цветных.

Пусть есть одинокая цветная клетка: сверху, снизу, слева и справа от неё либо пусто, либо край поля. Накрыть её нечем: доминошка занимает две клетки, и вторая её половина обязательно легла бы на пустое место, а это запрещено. Значит такая картинка не собирается, и программа именно так и объясняет отказ.

Наблюдение 3. Порядок разбора не важен

Это следует из первого наблюдения. Раз снятие одной доминошки никогда не мешает снять другую, порядок можно выбирать любой: что не убрано сейчас, спокойно уберётся потом.

Значит если просто убирать всё, что убирается, пока это возможно, результат всегда будет один и тот же - максимум того, что вообще можно разобрать. Это удобно: не нужно гадать, с чего начинать, и не нужно опасаться, что неудачное начало заведёт в тупик.

Главный вывод

Картинка собирается доминошками тогда и только тогда, когда разбор убирает с неё все цветные клетки. А если он их убрал, то обратный порядок разбора - готовый ответ: он даёт ровно ту картинку, которую требовалось получить.

5. Режим 1: когда доминошек ограниченное количество

Здесь мало разобрать картинку - нужно ещё уложиться в заданный набор. Сначала проверяется, собирается ли она вообще, как описано выше. Если нет, программа сразу пишет причину. Если собирается, начинается подбор порядка разбора, который влезает в набор: на каждом шаге программа смотрит, что сейчас можно убрать, берёт подходящий вариант и идёт дальше. Если получился тупик, перебор откатывается назад и пробует другой путь. Чтобы не считать одно и то же по многу раз, уже просмотренные положения запоминаются.

Проверка по количеству доминошек

Прежде чем запускать перебор, есть смысл посмотреть на картинку и на выданный набор. Каждая цветная клетка показывает цвет той доминошки, которая накрыла её последней. Значит за каждую цветную клетку отвечает какая-то доминошка того же цвета, а одна доминошка закрывает всего две клетки, поэтому отвечать больше чем за две клетки она не может.

Отсюда получается простое условие. **Клеток цвета с на картинке не может быть больше, чем $2 \cdot a_c$, где a_c — сколько доминошек цвета с выдано.** Если клеток больше, картинку не собрать, и это видно сразу, без единого шага перебора.

Важно, что это условие **необходимое, но не достаточное**. Оно отсекает заведомо безнадёжные случаи, но если проверка пройдена, картинка ещё не обязана собираться: доминошек может хватать по счёту, а разложить их так, чтобы получилась именно эта картинка, всё равно не выйдет. Поэтому после неё работают остальные проверки и перебор.

Просто жадно брать любую доминошку сверху нельзя, так как есть случаи на которых легко зайти в тупик:

Пусть на поле такая фигура, а доминошек выдано ровно две:

```
. 1 1
1 1 .
```

Если пожадничать и взять посередине вертикальную пару, фигура развалится на две одиночные клетки по краям, и накрыть их одной оставшейся доминошкой не выйдет. Программа доходит до этого тупика, откатывается и берёт две горизонтальные пары - они закрывают всё ровно за два хода. Если же ни один порядок в набор не влезает, программа сообщает: картинка собирается, но не из того, что выдано.

6. Режим 2: порядок задан заранее

Здесь последовательность менять нельзя и класть нужно все доминошки. Разбор идёт строго задом наперёд: последняя в списке доминошка клалась последней, значит и убирается первой - перебираются её возможные места. Затем предпоследняя и так далее. Место подходит, если под доминошкой нет пустых клеток и все неразобранные клетки там её цвета. Разобранные клетки не мешают, они подходят под любой цвет. Когда список кончился, проверяется, что цветных клеток не осталось.

Доминошки, которых не видно. Раз класть нужно все, какая-то из них может оказаться полностью закрашена теми, что легли позже, и на картинке её не будет вообще. При разборе это выглядит как место, где все клетки уже разобраны: доминошка как бы кладётся, но потом целиком исчезает под другими. Это разрешено, и такой случай отдельно проверяется в тестах.

Когда ответ «невозможно». Если для очередной доминошки не нашлось места или если места нашлись, но в конце на картинке остались цветные клетки, значит эта последовательность такую картинку не даёт.

7. Режимы 3 и 4: подсчёт числа картинок

Здесь вопрос другой: не «как сложили», а «сколько всего разных картинок может получиться». Разница между режимами простая. В четвёртом порядок цветов задан, а в третьем задан набор, и порядок выбирается свободно - а он важен, потому что доминошка, положенная позже, закрашивает те, что лежали раньше. Поэтому в третьем режиме перебираются ещё и порядки.

Сначала точный подсчёт

Главный приём: хранятся не ходы, а **сами получившиеся картинки**, и одинаковые сразу отбрасываются.

Проще всего показать на маленьком примере. Поле два на два, кладётся одна доминошка. Мест всего четыре, значит и картинок четыре. Теперь кладётся вторая: каждая из этих четырёх картинок берётся, и вторая доминошка ставится в каждое из четырёх мест. Действий получается шестнадцать, а разных картинок только девять - часть вариантов даёт один и тот же результат. Например, если положить вторую доминошку ровно туда же, куда первую, картинка вообще не изменится.

Именно поэтому перебор не разрастается: одинаковые картинки не размножаются дальше, а склеиваются в одну. В третьем режиме рядом с картинкой хранится ещё и остаток набора - «такая картинка получена, красных осталось ноль, синих одна», иначе неизвестно, что ещё предстоит положить.

Если картинок слишком много

На большом поле мест для доминошки уже больше сотни, и картинок становятся сотни миллионов. Хранить их нигде, программа просто зависнет. Поэтому стоит ограничение: как только картинок набралось больше него, точный подсчёт прекращается и включается оценка.

Работает это как со сбором наклеек: покупаешь пачки наугад и смотришь, что внутри. Если одни и те же наклейки попадают снова и снова - значит видов немного и почти все уже собраны. А если почти каждая пачка приносит новую наклейку - значит несобранных ещё много.

Программа делает то же самое: много раз раскладывает доминошки наугад и запоминает, какие картинки получились и сколько раз каждая. Затем смотрит, сколько картинок попало ровно один раз и сколько ровно два, и по этому оценивает, сколько осталось не увидено. Формула готовая, её предложил Чао в 1984 году:

$$D + f1 \cdot (f1 - 1) / (2 \cdot (f2 + 1))$$

Здесь D - сколько разных картинок встретилось всего, f_1 - сколько из них попались ровно один раз, f_2 - ровно два раза. Смысл простой: чем больше картинок попало всего по одному разу, тем больше их осталось за кадром.

Ответ показывается так: «**примерно 400 000 000, но точно не меньше 39 999**». Второе число честное - эти картинки действительно встретились, меньше их быть не может. Первое показывает порядок величины и специально округляется до нескольких первых цифр, чтобы оценку не приняли за точный ответ.

Слабое место у такой оценки тоже есть. Она хорошо работает, когда картинки в выборке повторяются. Если же почти каждая случайная раскладка даёт новую картинку, формула показывает только порядок величины, а не точное число. Но и это полезно, а нижняя граница остаётся строгой в любом случае.

8. Скорость

Сначала стоит посчитать, сколько вообще есть мест для доминошки на поле со стороной n . Вдоль строки её можно положить $n-1$ способом, строк всего n , и столько же вариантов, если класть поперёк. Итого $2 \cdot n \cdot (n-1)$ мест: например, на поле три на три это 12, а на поле восемь на восемь уже 112.

- Режимы 1 и 2. На каждом шаге перебираются все эти места, а шагов не больше, чем клеток на поле. Перебор мог бы разрастись, но он сильно сокращается за счёт того, что уже просмотренные положения запоминаются и не считаются заново. На небольших полях из условия ответ появляется сразу.
- Точный подсчёт в режимах 3 и 4. Здесь работа зависит от числа разных картинок, и как только их становится слишком много, производится оценка.
- Оценка считается быстро даже на большом поле, потому что число случайных раскладок задаётся заранее и от размера поля не зависит.

9. Тестирование

В проекте 33 проверки, они запускаются одной командой. Кроме мелких случаев есть три большие сверки, на которые опирается основная уверенность в правильности.

- **Сверка по всем картинкам подряд.** На поле два на два с двумя цветами всего 81 картинка. Перебором находится, какие из них вообще собираются, и результат сверяется с ответом решателя по каждой. Расхождений нет ни одного.
- **Обратная проверка ответа.** Каждый найденный порядок ходов раскладывается обратно на пустом поле и сравнивается с исходной картинкой. Так проверяются 200 случайных картинок, заранее собранных случайными ходами.
- **Сверка подсчёта.** Быстрый способ подсчёта сравнивается с перебором в лоб на маленьких полях - числа обязаны совпасть. Отдельно проверяется, что на большом поле включается оценка.

Отдельными тестами закрыты каверзные случаи: когда важен порядок разбора, одинокая клетка, нехватка доминошек, перекрытия, полностью закрашенная доминошка и тупик, из которого нужно откатиться, а также проверка по количеству доминошек — и когда их не хватает, и когда хватает ровно впритык.

10. Важная оговорка

«Лишние»* доминошки разрешено не класть в первом режиме, задача - уложиться в набор, так как их просто можно положить на поле одну на одну на клетки, которые не будут являться фоном в итоговом разложении.

**Лишними названы домино, которые не участвуют в итоговом разложении*

11. Итоги и что можно доделать

Сделаны все четыре режима, окно с пошаговым показом решения, версия для консоли и проверки. Решатель написан на голем Python и ничего не требует доустанавливать - библиотеки нужны только для графического интерфейса.

Что можно доделать дальше: научиться оценивать, насколько можно доверять оценке, ускорить подсчёт за счёт хранения поля числом вместо таблицы и добавить сохранение картинки в файл, чтобы не вводить её заново.

Блок ИИ:

Для реализации проекта использовалась ИИ модель Claude Opus. В её задачи входила реализация кода на Python и разработка графического интерфейса. Всю логику решения задачи придумал лично я. Из языков программирования знаю только C++ и имею опыт только в решении олимпиадных задач. Я был очень заинтересован в реализации проекта, который будет не стыдно показать, поэтому прибег к помощи ИИ в реализации моей идеи. После реализации проекта я загорелся желанием изучить Python(новый для меня язык), так как очень понравилось работать над собственным приложением.